



Algorithmique : Devoir

Professeur encadrant : M. Roupin

Année universitaire : 2023 – 2024

Naoufal Benamar

└ Algorithmique ─

└ Devoir ─

└ Objectifs et résultats préliminaires ─

1) Supposons que A est une matrice réelle inversible carrée de taille n .

- Pour montrer que $A^T A$ est symétrique, il faut montrer que :

$$(A^T A)^T = A^T A$$

$$(A^T A)^T = A^T (A^T)^T = A^T A \text{ donc } A^T A \text{ est symétrique.}$$

- Pour montrer que $A^T A$ est définie positive ($\forall x \neq 0 \in \mathbb{R}^n, x^T A^T A x > 0$)

Soit $x \neq 0 \in \mathbb{R}^n$,

On pose $y = Ax$ avec $y \in \mathbb{R}^n / y = (y_1, \dots, y_n)$

Puisque x est non nul et A est inversible alors $y \neq 0$
avec $x^T A^T A x = (Ax)^T A x = y^T y$

On sait que $y^T y = \sum_{i=1}^n y_i^2 > 0$ car $y \neq 0$

$$\text{Alors } x^T A^T A x > 0$$

Donc $A^T A$ est définie positive.

$$2) \quad (A^T A)^{-1} A^T A = I_n \quad (\text{car } X^{-1} X = I) \quad (A \text{ inversible} \Rightarrow \text{donc } A^T A \text{ inversible})$$

$$A \cdot (A^T A)^{-1} A^T = I_n \quad (\text{car } (A^T A)^{-1} A^T A = I_n \Rightarrow (A^T A)^{-1} A^T = A^{-1} \Rightarrow A(A^T A)^{-1} A^T = I_n)$$

Puisque: $A \cdot (A^T A)^{-1} A^T = (A^T A)^{-1} A^T A = I_n$

Alors: $A^{-1} = (A^T A)^{-1} A^T \quad (i)$

(i) montre qu'au lieu de calculer l'inverse d'une matrice quelconque, on peut calculer l'inverse d'une matrice définie positive et faire une transposition et deux multiplications.

$$3) \quad A^T A \text{ définie positive} \Leftrightarrow \forall x \in \mathbb{R}^n \setminus \{0\}, x^T A^T A x > 0$$

On pose $x = \begin{pmatrix} y \\ z \end{pmatrix}$ donc $x^T = (y^T \ z^T)$

$$x^T A^T A x = (y^T \ z^T) \begin{pmatrix} B & C^T \\ C & D \end{pmatrix} \begin{pmatrix} y \\ z \end{pmatrix} = (y^T \ z^T) \begin{pmatrix} By + C^T z \\ Cy + Dz \end{pmatrix}$$

$$= y^T B y + y^T C^T z + z^T C y + z^T D z$$

$$= y^T B y + y^T B B^{-1} C^T z + z^T C B^{-1} B y + z^T C B^{-1} B C^T z$$

$$+ z^T D z - z^T C B^{-1} C^T z$$

$$= (y + B^{-1} C^T z)^T B (y + B^{-1} C^T z) + z^T (D - C B^{-1} C^T) z$$

Alors on choisit x tel que $y + B^{-1} C^T z = 0$

Donc $x^T A^T A x > 0 \Leftrightarrow z^T (D - C B^{-1} C^T) z > 0$

Alors $A^T A$ définie positive $\Leftrightarrow S = D - C B^{-1} C^T$ définie positive

4)

$$\begin{aligned}
& \begin{pmatrix} B^{-1} + B^{-1} C^T S^{-1} C B^{-1} & -B^{-1} C^T S^{-1} \\ -S^{-1} C B^{-1} & S^{-1} \end{pmatrix} \begin{pmatrix} B & C^T \\ C & D \end{pmatrix} \\
&= \begin{pmatrix} I + B^{-1} C^T S^{-1} C - B^{-1} C^T S^{-1} C & B^{-1} C^T + B^{-1} C^T S^{-1} C B^{-1} C^T - B^{-1} C^T S^{-1} D \\ -S^{-1} C + S^{-1} C & -S^{-1} C B^{-1} C^T + S^{-1} D \end{pmatrix} \\
&= \begin{pmatrix} I & B^{-1} C^T + B^{-1} C^T (S^{-1} C B^{-1} C^T - S^{-1} D) \\ 0 & S^{-1} (D - C B^{-1} C^T) \end{pmatrix} \\
&= \begin{pmatrix} I & B^{-1} C^T - B^{-1} C^T S^{-1} (D - C B^{-1} C^T) \\ 0 & S \cdot S^{-1} \end{pmatrix} \\
&= \begin{pmatrix} I & B^{-1} C^T - B^{-1} C^T S^{-1} S \\ 0 & I \end{pmatrix} \\
&= \begin{pmatrix} I & 0 \\ 0 & I \end{pmatrix} = I
\end{aligned}$$

4) suite

$$\begin{aligned}
 & \begin{pmatrix} B & C^T \\ C & D \end{pmatrix} \begin{pmatrix} B^{-1} + B^{-1} C^T S^{-1} C B^{-1} & -B^{-1} C^T S^{-1} \\ -S^{-1} C B^{-1} & S^{-1} \end{pmatrix} \\
 &= \begin{pmatrix} I + C^T S^{-1} C B^{-1} - C^T S^{-1} C B^{-1} & -C^T S^{-1} + C^T S^{-1} \\ C B^{-1} + C B^{-1} C^T S^{-1} C B^{-1} - D S^{-1} C B^{-1} & -C B^{-1} C^T S^{-1} + D S^{-1} \end{pmatrix} \\
 &= \begin{pmatrix} I & 0 \\ C B^{-1} - S^{-1} C B^{-1} \underbrace{(D - C B^{-1} C^T)}_{=S} & S^{-1} \underbrace{(D - C B^{-1} C^T)}_{=S} \end{pmatrix} \\
 &= \begin{pmatrix} I & 0 \\ C B^{-1} - C B^{-1} & I \end{pmatrix} \\
 &= \begin{pmatrix} I & 0 \\ 0 & I \end{pmatrix} = I
 \end{aligned}$$

On a donc $(A^T A)^{-1} (A^T A) = (A^T A) (A^T A)^{-1} = I$

Alors :

$$(A^T A)^{-1} = \begin{pmatrix} B^{-1} + B^{-1} C^T S^{-1} C B^{-1} & -B^{-1} C^T S^{-1} \\ -S^{-1} C B^{-1} & S^{-1} \end{pmatrix}$$

↳ Algorithme et analyse de sa complexité

Ici, nous supposons que les multiplications sont effectuées à l'aide de la multiplication standard.

Pour établir la relation de récurrence entre $C(n)$ et $C(\frac{n}{2})$, on utilisera le théorème du maître qui stipule :

• Pour $C(n) = a C(\frac{n}{b}) + O(n^d)$, le terme général est :

$$C(n) = \begin{cases} O(n^d) & \text{si } d > \log_b a \\ O(n^d \log n) & \text{si } d = \log_b a \\ O(n^{\log_b a}) & \text{si } d < \log_b a \end{cases}$$

On a : $C(n) = 2C(\frac{n}{2}) + \alpha n^3 + \beta n^2$

$2C(\frac{n}{2})$ car on a deux calculs récursifs de $2C(\frac{n}{2})$

Or : $\alpha n^3 + \beta n^2 = O(n^3)$ car αn^3 domine βn^2

On a donc : $C(n) = 2C(\frac{n}{2}) + \alpha n^3$

* α représente le nombre de multiplications, il est égal à 6.

* β représente le nombre de transpositions / additions / soustractions car ces trois opérations ont une complexité quadratique.

En récursif cela donne :

$$\begin{aligned}
 C(n) &= 2^2 C\left(\frac{n}{2^2}\right) + 2 \left(\frac{n}{2}\right)^3 + \alpha n^3 \\
 &= 2^k C(2) + 2^{k-1} \left(\frac{n}{2^{k-1}}\right)^3 + \dots + \alpha n^3 \\
 &= p n^{\log_2(2)} + \alpha n^3 \sum_{i=0}^{k-1} 2^i \left(\frac{1}{2^i}\right)^3 \quad (C(2) = p = \text{constante}) \\
 &= pn + \alpha n^3 \sum_{i=0}^{k-1} \left(\frac{1}{4}\right)^i \\
 &= pn + \alpha n^3 \left(\frac{1 - \left(\frac{1}{4}\right)^k}{1 - (1/4)} \right) \\
 &= pn + \alpha n^3 \times \frac{4}{3} \left(1 - \left(\frac{1}{4}\right)^k\right) \\
 &= pn + \alpha' n^3 \left(1 - \frac{1}{n^2}\right) \quad \left(\left(1 - \left(\frac{1}{4}\right)^{\log_2(n)}\right) = \left(1 - n^{-\log_2(4)}\right) = \left(1 - n^{-2}\right) \right) \\
 &= pn + \alpha'' n^3 \\
 &= O(n^3)
 \end{aligned}$$

$\alpha' = \frac{4}{3} \alpha$
 $\alpha'' = \alpha' \left(1 - \frac{1}{n^2}\right) \xrightarrow{n \rightarrow \infty} \alpha'$

La complexité en temps (nombre d'opérations) est donc égale à n^3 .

Taille des données en entrée : $T = O(n^2 \log(N)) = O(n^2)$
 (N : plus grand scalaire)
 (car $\log(N)$ négligeable devant n^2)

On a donc : $T = O(n^2)$ et $C(n) = O(n^3)$

La complexité augmente de façon polynomiale donc l'algorithme est efficace mais que pour des petites matrices. Cela peut vite s'avérer très gourmand en ressource pour des grandes matrices.